

Programação Orientada a Objetos: Estruturação de Classes e Métodos em Python

13 de abril de 2026

[Murilo G. Oliveira] | 0009-0003-7233-4763

[Gabriel T. O. Nakamura] | 0009-0002-9840-0039

Orientador - Eduardo F. Miranda | 0000-0003-1200-794X

1 Introdução

O paradigma da Programação Orientada a Objetos (POO) representa um marco na engenharia de software, permitindo a criação de sistemas modulares e reutilizáveis. Conforme abordado por Scheffer (2026), a linguagem Python facilita a implementação desses conceitos através de uma sintaxe clara para a definição de classes. Os experimentos práticos que fundamentam esta análise foram validados em ambiente interativo, cujo código pode ser consultado via Google Colaboratory através da URL: <https://colab.research.google.com/drive/1nTKvRXr3Agjxq7C3y5D3PmxW-9ZfxmKI?usp=sharing>.

2 Fundamentos: Classes e Objetos

Na visão de Scheffer (2026), uma classe deve ser compreendida como um projeto ou molde que descreve o comportamento e os dados de um tipo de objeto.

Destaque Conceitual: A instanciação é o processo de criar um objeto real a partir de uma classe. Cada objeto possui seus próprios atributos, mas compartilha a estrutura lógica definida pelo seu molde (SCHEFFER, 2026).

3 Implementação Técnica em Python

A manipulação de classes em Python envolve a utilização do método construtor e a definição de comportamentos específicos. A Tabela 1 sintetiza os elementos fundamentais da POO:

3.1 Exemplo de Código Estruturado

Abaixo, observa-se a aplicação prática da criação de uma classe, utilizando o encapsulamento de lógica para controle de atributos e métodos de instância:

Tabela 1: Elementos Estruturais da POO

Componente	Função	Exemplo
Atributos	Definem o estado do objeto	<code>self.nome, self.vida</code>
Métodos	Definem o comportamento	<code>def tomar_dano(self)</code>
<code>__init__</code>	Método construtor	Inicialização de variáveis

```
class Personagem:
    def __init__(self, nome, vida):
        self.nome = nome
        self.vida = vida

    def atualizar_status(self, novo_hp):
        self.vida = novo_hp
        print(f"Status de {self.nome} atualizado para {self.vida} HP.")
```

4 Conclusão

A transição para o uso de POO é essencial para o desenvolvimento de aplicações robustas. Scheffer (2026) conclui que, ao dominar a criação de classes e métodos, o desenvolvedor ganha a capacidade de gerenciar sistemas complexos com maior legibilidade e eficiência, preparando o código para futuras expansões.

Referências

1. SCHEFFER, Vanessa Cadan. **Linguagem de Programação: Classes e Métodos**. Livro Digital. Valinhos: Editora Educacional, 2026.
2. GUIMARÃES, M. G.; NAKAMURA, G. T. O. **Linguagem de Programação: Classes e Métodos**. Google Colaboratory, 2026. Disponível em: <https://colab.research.google.com/drive/1nTKvRXr3Agjxq7C3y5D3PmxW-9ZfxmKI?usp=sharing>. Acesso em: 17 de abril de 2026.